# Chapter 13. Machine Learning/Artificial Intelligence

## 13.0 Introduction

The notion of "artificial intelligence" was envisioned in science fiction for some years before the first software developments in that direction. Today we still have AI in fiction, like Jarvis, Iron Man's helmet AI. But we also have AI in real life, coming to popular attention starting with ChatGPT in the early 2020s. It's actually been in the works for three quarters of a century.

There are two main approaches to AI: machine learning (ML) and coding. Coding toward AI was one of the factors that led to the creation of the Lisp language in the late 1950s. The goal of coding AI was defined as Artificial General Intelligence, or AGI: an AI that could pass the Turing Test of interacting exactly as a human would. Of course, "intelligence" has many different definitions...

Machine learning (ML), based on neural nets, has been around since at least the 1980s. ML doesn't necessarily aim for AGI, just to perform classification and generation tasks better—or at least faster—than humans. Unlike coded solutions that can be understood by humans, neural nets are based on training. Training data is fed into the AI, which ingests and stores it, for use when interacting with a user. Over time developers connected several neural nets in a row, resulting in the term *deep learning*.

This training process typically consumes massive amounts of computing power, so only small datasets can be trained on a typical mid-2020s desktop computer. Inferencing (using the generated model) takes less overhead, and can be done locally, though rendering (particularly images) is often best done in the cloud. This chapter will focus primarily on the "using" side of things.

One of the original examples of ML is classification. Suppose you want to generate software to tell if a given photographic image contains a human being. The developer feeds in a large number of examples that have been manually labeled (e.g., "person," "no person") and the software outputs a neural net that "just works"—you don't necessarily know *how* it works, only *how well* it works.

You can think of these classifiers as decision trees. For example, I need to get to a meeting after a night of hanging out late with friends. If it is raining, then if I can find my umbrella I can walk, or if I can find my car keys I can drive, or I must ride the bus or a rideshare. If it's not raining, I can also walk. Figure 13-1 is a simplified example, of course. Some AI software will print out the decision tree it has generated.

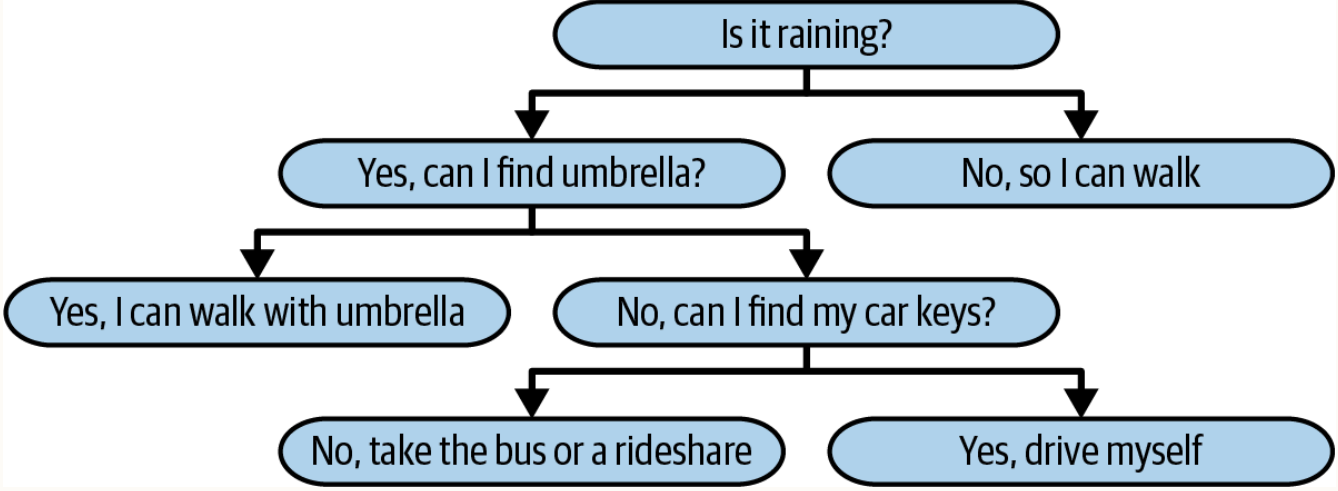



Figure 13-1. Simple decision tree example

Another kind of ML, popularized by (but not exclusive to) ChatGPT, is generative AI. You can feed in all the source code on GitHub and pirated copies of books, and generative AI can tell you how to write any kind of program that was described in its input, and explain how it works, based on its readings. Because these involve massive amounts of data, the generated networks are called large language models (LLMs). As Tim O'Reilly correctly points out in a commentary on the 2023 Biden Administration's AI directive, the use of books and artworks (without consent) presumably violates the copyright on the books, although the source code in public GitHub repos is generally assumed to be fair game. The ancient computing adage GIGO applies as much to AI as to any computation: *garbage in, garbage out*. An AI fed political misinformation may report as "facts" events that never happened, or as "heroes" those who tried to overthrow a legitimate government. Further, some AIs will hallucinate—return a dreamed-up answer—when they don't have a clue. Such errors, of course, can have serious consequences in the real world.

At best, this type of AI can often do tasks better and/or faster than humans, which has the potential to lead to massive job losses in many fields over the coming years. People may stop buying books (like mine!) when they can just ask ChatGPT—though the same fear was raised when Google started indexing everything including GitHub repos. It can be a benefit to some, as in a coding assistant for developers. It can also save lives. Here's one example out of many using classification in medical care. St. Michael's Hospital in Toronto is a regional trauma center, that is, the hospital where EMTs know to take people banged up in serious car crashes, major accidents, and those injured with malevolent intent (e.g., a crowbar to the head). In addition to examination by a qualified emergency room doctor, incoming brain trauma patients will have brain scans. Patients will partly be triaged by an AI application to see if their injuries require immediate treatment (without which they will shortly die) or whether they will survive if they wait while the more serious injuries are treated. The training for this involves many, many scans that have been tagged with their actual outcomes, along with input from a number of ER doctors with experience in different patient ages, ethnicities, kinds of injuries, etc. It saves lives.

It's sometimes said that machine learning and deep learning have to be done in C++ for efficiency or in Python for the wide availability of software. While these languages have their pros and cons (and their passionate devotees), this is the *Java* Cookbook, so this chapter focuses on a few toolkits for doing ML-type AI in Java. There isn't space for examples of all the good Java-based AI software out there. Some can run on your desktop, some with support for NVIDIA's CUDA API[1] for faster GPU-based processing, so there is no reason to avoid using Java for ML. Table 13-1, lists packages that are currently maintained and have a decent reputation among users.

Table 13-1. Some Java machine learning packages

| Library name | Description | Info URL | Source URL |
|---|---|---|---|
| ADAMS | Workflow engine for building/maintaining data-driven, reactive workflows; integration with business processes | https://adams.cms.waikato.ac.nz | https://github.com/waikato-datamining/adams-base |
| Deep Java Library | Amazon's ML library | https://djl.ai | https://github.com/awslabs/djl |
| Deeplearning4j | DL4J, Eclipse's distributed deep-learning library; integrates w/ Hadoop and Apache Spark | https://deeplearning4j.org | https://github.com/eclipse/deeplearning4j |
| ELKI | Data mining toolkit | https://elki-project.github.io | https://github.com/elki-project/elki |
| LangChain4j | Unified interface to many popular third-party AI toolkits | https://docs.langchain4j.dev | https://github.com/langchain4j/langchain4j |
| Mallet | ML for text processing | https://mimno.github.io/Mallet/index | https://github.com/mimno/Mallet.git |
| Spring AI | Unified interface to many popular third-party AI toolkits | https://spring.io/projects/spring-ai | https://github.com/spring-projects/spring-ai |
| TensorFlow | Open source framework for creating and deploying ML models | https://www.tensorflow.org | https://github.com/tensorflow/tensorflow |
| Tribuo | Unified interface to many popular third-party ML libraries | https://github.com/oracle/tribuo | https://github.com/oracle/tribuo |
| Weka | ML algorithms for data mining; tools for data preparation, classification, regression, clustering, association rules mining, and visualization | https://www.cs.waikato.ac.nz/ml/weka/index.html | https://svn.cms.waikato.ac.nz/svn/weka/trunk/weka |

# 13.1 Some Major AI Software

There are indeed many AI options for Java developers. I've highlighted some of the major ones in the following sections, though most of my examples focus on LangChain4j, which appears to be among the more widely used offerings. You may also refer to Eugen Parschiv's list of Java AI software packages. Also, AI is constantly changing, so you should always check out the online documentation for the latest version of whatever framework you are using.

NOTE

DeepSeek-R1 was released too close to this book's publication for coverage.

---

## LangChain4j

LangChain4j offers a frontend to numerous API toolkits and datasets. It was self-admittedly somewhat patterned after the Python-based langchain, but shaped with ideas from several other toolkits, and designed specifically as a Java API, not a translation (it doesn't include any Python code). There are examples of LangChain4j in this chapter.

## Spring AI

Spring AI offers a frontend to numerous API toolkits and datasets. It can leverage the many other Spring projects, including dependency injection, security, and the like. Spring also offers a convenient tool for building a "starter app," available online at *https://start.spring.io* (be sure to specify Spring AI as a dependency) or interactively via a Spring-provided CLI tool. I don't give examples of Spring AI due to space limitations, but Spring API provides similar functionality to the LangChain4j examples.

## TensorFlow

One of the earliest widely used libraries is Google's TensorFlow. Written in Python and C++, TensorFlow can, like most AI toolkits, use GPUs, which provide faster computing for certain types of problems than do the general-purpose CPUs found in AMD/Intel-based computers. A programming language called CUDA is one of several for this purpose, and is the one used by TensorFlow. Although not written in Java, TensorFlow has interfaces for Java and other languages. See also the Wikipedia article on TensorFlow.

## Oracle Tribuo

Oracle's Tribuo aims to be a comprehensive Java toolkit for AI. As its website introduces it, Tribuo:

> *provides tools for classification, regression, clustering, model development, and more. It provides a unified interface to many popular third-party ML libraries like xgboost and liblinear. With interfaces to native code, Tribuo also makes it possible to deploy models trained by Python libraries (e.g. scikit-learn, and pytorch) in a Java program.*

Its source code is available in the Tribuo GitHub repository. The name comes from Latin roots meaning to assign or apportion; putatively from the same roots as the English word "tribute."

## Other

Although it's not a Java API, Luma Labs makes several interesting online-only applications, including Genie, which creates 3D models from text prompts, and Dream Machine, which creates videos from text.

The issue of ethical considerations around AI is the subject of university-level courses, so this will just touch on a few of them.

The first is copyrights. It is widely believed that some of the large data model-based generative AIs may have been trained on works that are copyrighted but used without permission. It's not possible to prove or disprove this idea, because the training data is usually not available (and most people couldn't download it anyway given how massive the datasets are). Obviously, use of copyrighted material for training a generative AI for public use is a violation of copyright, because the information in the copyrighted work will be regurgitated, albeit blended with other similar works. That's just how generative AI operates.

There are solutions; Google researchers have tricked ChatGPT into giving up some of its training data. Of course, this raises privacy issues if information with personally identifiable data was used in training (and it was, according to the paper). Among visual artists, there have been several "image poisoners" who tweak an image so it looks like one thing to an AI but still has its original meaning to humans viewing it as an image.

Another issue is reliability. Consider a medical app that tries to analyze scanned images of neoplasms to see if they are benign (harmless) or malignant (cancerous). Andrew Tait in the Learning Tree Course "Introduction to AI" compares two variants of one such decision-tree-based classification app. The lighter boxes show the correct diagnoses. The model was trained on thousands of scan images (far more than a human radiologist is trained on). The images were classified manually by trained medical personnel, and corrected with regard to the actual patient outcomes where available.

When tested, one model gives 97% correct classifications, the other only 94%. Is the first one better? One might at first say yes. But look at the "false negative" section—the "more accurate" one will send three patients out of every one hundred home to die, thinking they are cancer-free. The "less accurate" one will only send one patient home oblivious to the cancer destroying their body. It will set five to worrying, but they will probably be OK once they get biopsied. Both are probably about as good as, or better than, a trained radiologist. Which one is better? I'd certainly rather be tested by the second one. Figure 13-2 compares the two models.

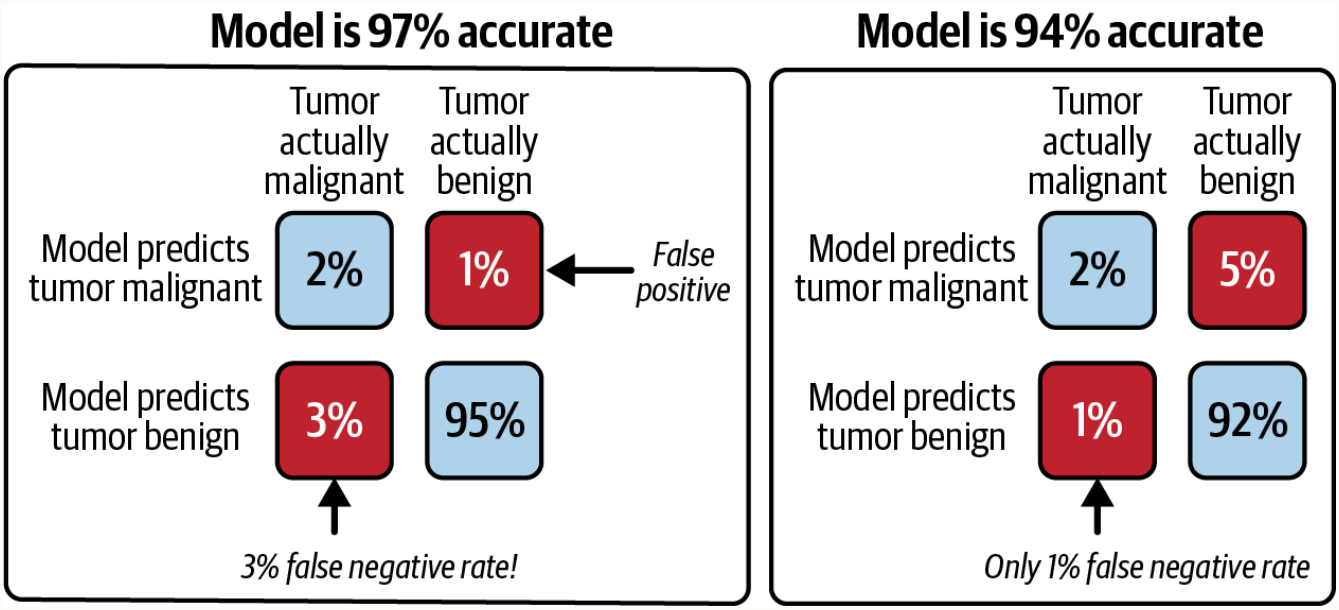



**Figure 13-2.** More accuracy isn't always the best indicator

AIs are also prone to hallucinations, when they make up answers, or create images of people with the wrong number of fingers or even the wrong number of heads. At the present stage of AI, all of its output needs to be screened by a competent adult human before use.

Yet another issue is bias. Suppose that different ethnic groups in general have different work ethics or financial abilities (this may or may not be true, but assume it is for this discussion). Suppose further that a financial AI model is trained primarily on white middle class people because their data is more readily available to the financial institution developing the model. Is it fair to use this model to make financial decisions (e.g., accept/reject loan applications) about people from any minority background or orientation? I'd say probably not.

Let's not even consider the possibility of massive job losses as AI models get as good as humans in some fields, and are cheaper; the arguments for and against this being likely could fill an entire chapter in a book this size.

There's even a hidden power connection. The electrical energy to power the computers used to train some of the very largest models could probably run a small city for a week; this seems to have implications for both climate activists and poverty activists to ponder. We must leave such issues for others. Further, if millions of people start using AI for spell-checking or web search, they're going to be wasting huge swathes of energy.

One interesting reference on the subject is Mustafa Suleyman's The Coming Wave. Suleyman is the co-founder of DeepMind (which was sold to Google) and Inflection.ai, so he knows a thing or two about AI.

There are many, many ethical questions raised by the increasing use of artificial intelligence. There are, at present, far fewer answers than questions.

---

### See Also

Classification and generative AI have become extremely hot topics in the year or so leading up to this writing. There are predictions of major job losses as AIs continue to improve their ability to perform many tasks better, faster, and cheaper than humans. Once the AIs start rewriting themselves on the fly, all bets are off. Meanwhile, we do what we can with what we have.

---

NOTE

The examples in the following recipes use these constants, defined here to ensure they're used consistently:

```
public static final String TEXT_PROMPT =
 "What are the great themes in literature?";
public static final String IMAGE_PROMPT =
 "Charles Darwin in modern Toronto, cartoon style";
```

To run the OpenAI-based models in this chapter, you need to obtain an API key from the OpenAI signup page, and put a few dollars toward the cost of using the examples (most cost under a nickel to run, some under a penny). Those examples also use the method `getOpenAPIKey()`, which retrieves your required OpenAPI key from a text file in your home directory.

---

## 13.2 Using ChatGPT Directly

### Problem

You want to use ChatGPT directly, without bothering to learn any high-level Java AI toolkits.

### Solution

Use the `java.net` API for networking and the `Jackson` API for JSON formatting and parsing.

## Discussion

Like almost all online AI tools, ChatGPT can be accessed with a simple REST API using JSON-format requests and responses (REST is explained in Recipe 14.1).

Example 13-1 shows how one might approach this, using the Java API for networking and the Jackson library for XML formatting.

**Example 13-1. ai/src/main/java/chatgpt/ChatGptApiDemo.java**

```
final static String URL = "https://api.openai.com/v1/chat/completions";
final static boolean DUMP_RAW = false;

public static void main(String[] args) throws Exception {
 System.out.println(chatGPT(Constants.TEXT_PROMPT));
}

public static String chatGPT(String prompt)
   throws IOException, URISyntaxException {
 String apiKey = Constants.getOpenAPIKey();
 String model = "gpt-4o";
 HttpClient client = HttpClient.newHttpClient();

 URI uri = new URI(URL);

 // Send the request
 var request = new ChatGptRequest(model, "user", prompt);
 String json = request.toString();
 System.out.println("Sending this: " + json);

    HttpRequest webRequest = HttpRequest.newBuilder()
    .uri(uri)
    .header("Content-Type", "application/json")
    .header("Authorization", "Bearer " + apiKey)
    .POST(HttpRequest.BodyPublishers.ofString(json))
    .build();
 HttpResponse<String> response = null;
 try {
  response = client.send(webRequest, HttpResponse.BodyHandlers.ofString());
  System.out.println("Response code: " + response.statusCode());
  System.out.println("Response body: " + response.body());
 } catch (IOException | InterruptedException e) {
  e.printStackTrace();
 }

 // Get a response from ChatGPT
 int n = response.statusCode();
 switch(n) {
  case 429:
    System.out.println("No ChatGPT wants to listen to your chuntering.");
    String rah = null;
    if ((rah =
      response.headers().map().get("retry-after").getFirst()) != null) {
      System.out.println("Try again: " + rah);
    }
    System.out.println("Response body: " + response.body());
    return null;
  case 500: case 501: case 502:
```

```
      System.out.println("GPT Server error " + n + " " + response.body());
      return null;
    default:
      System.out.println("HTTP Status was " + n);
  }

  if (DUMP_RAW) {
    System.out.println("Response body: " + response.body());
    return "Answer dumped, no JSON parsing done.";
  } else {
    var mapper = new ObjectMapper();
    // Guard against fields added as GPT evolves
    mapper.configure(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES, false);
    ChatGptResponse resp =
      mapper.readValue(response.body(), ChatGptResponse.class);
    System.out.println(resp);
    return resp.choices[0].message.content;
  }
 }

record ChatGptRequest(String model, String role, String prompt) {
  public String toString() {
    return """
      {"model": "%s", "messages": [{"role": "%s", "content": "%s"}]}"""
      .formatted(model, role, prompt);
  }
}

/** These could be records or classes */
static class ChatGptResponse {
  public String id;
  public ChatGptChoice[] choices;
  public String object;
  public long created;
  public String model;
  public ChatGptUsage usage;
  public String system_fingerprint;

  public String toString() {
    var sb = new StringBuilder();
    for (ChatGptChoice comp : choices) {
      sb.append(comp).append('\n');
    }
    return sb.toString();
  }
}

static class ChatGptChoice{
  public int index;
  public ChatGptMessage message;
  public Object logprobs;
  public String finish_reason;
}

static class ChatGptMessage {
  public String role;
  public String content;
  public String refusal;
}
```

```
  static class ChatGptUsage {
    public int prompt_tokens;
    public Object prompt_tokens_details;
    public int completion_tokens;
    public Object completion_tokens_details;
    public int total_tokens;
  }
```

`main()` just calls our `chatGPT()` method with the query.

Using an `HttpURLConnection` (see HttpURLConnection example), we connect to the ChatGPT URL, providing an authentication key (for billing!) and setting the content type to JSON.

`ChatGptRequest` is our holder for all the pieces of a request, and will be crudely translated into JSON in its `toString()` method as we write it onto the output side of the connection, which we close to indicate that we're done sending.

ChatGPT uses the particular status code 429 indicating that it's busy or that you have been using too much computing; for that status code we try some error handling.

For a normal status of 200, the code optionally (based on `DUMP_RAW`) prints the results as raw JSON text, or maps it into Java objects defined after main, and extracts just the relevant portion (the message content).

The JSON parsing could be done just using `JSONObject` and `JSONArray`, but whether done that way or not, the response is dependent upon the message format returned by the other end.

# 13.3 Using ChatGPT via LangChain4j

## Problem

You want to get the results of a ChatGPT prompt with minimal effort.

## Solution

Use LangChain4j.

## Discussion

The use of a higher-level API can greatly simplify coding. LangChain4j is a higher-level API that eases the cost of developing AI-based software, and provides access to a wide range of large language models (LLMs). To start, one need only add these dependencies to *pom.xml* or *build.gradle* (adjusting the version to the latest one available):

```
<properties>
    <langchain4j-version>0.31.0</langchain4j-version>
</properties>
```

```
<dependency>
    <groupId>dev.langchain4j</groupId>
    <artifactId>langchain4j</artifactId>
    <version>${langchain4j-version}</version>
</dependency>
```

```
<!-- A dependency per provider that you want to use: here, openai -->
<dependency>
    <groupId>dev.langchain4j</groupId>
    <artifactId>langchain4j-open-ai</artifactId>
    <version>${langchain4j-version}</version>
</dependency>
```

```
<dependency>
    <groupId>org.slf4j</groupId>
    <artifactId>slf4j-api</artifactId>
    <version>2.0.13</version>
</dependency>
```

Then, obtaining a query result using the same prompt as used in Recipe 13.2 can be as simple as Example 13-2.

**Example 13-2. ai/src/main/java/langchain4j/LangChain4JSimplePrompt.java**

```
/**
 * Simple demo of a chat-gpt text prompt and response.
 */
public class LangChain4JSimplePrompt {

    public static void main(String[] args) {

        String apiKey = Constants.getOpenAPIKey();
        ChatLanguageModel model = OpenAiChatModel.withApiKey(apiKey);

        String results = model.generate(Constants.TEXT_PROMPT);

        System.out.println(results);
    }
}
```

When run, this outputs a short outline of the great themes, as requested.

# 13.4 Making an AI Service with LangChain4j

## Problem

You want a higher-level access to AI, such as using an AI to scan through text looking for certain fields.

## Solution

Use a service interface returning a POJO containing those fields.

## Discussion

For this we define a service interface, which LangChain4j will implement. The example service is to convert plain-text name-and-address information into a structured form, but this mechanism can be used for a wide range of service types. Invoking the method will populate as many of the `Contact` POJO's fields as possible. Example 13-3 shows using this feature to solve the age-old problem of converting plain text with a name, address, etc., into a "contact" in-

formation structure with those values as fields, to be imported into whatever contact manager you use. This example also shows a more general method of constructing the "AI" object.

**Example 13-3. ai/src/main/java/langchain4j/LangChain4JContactFinder.java**

```
public class LangChain4JContactFinder {

 record Contact(        ❶
  String firstName,
  String lastName,
  String company,
  String street,
  String city,
  String state,
  String code) {

  @Override
  public String toString() {    ❷
   return String.format("""
     Contact {
      name    : %s %s
      company : %s
      street  : %s
      place   : %s, %s %s
     }""",
     firstName, lastName, company,
     street, city, state, code);
  }
 }

 interface ContactFinder {      ❸

  @UserMessage("Extract information about a person from {{it}}")
  Contact extractContactFrom(String text); ❹
 }

 public static void main(String[] args) {

  ChatLanguageModel chatLanguageModel = OpenAiChatModel.builder()
     .apiKey(Constants.getOpenAPIKey())
     .responseFormat("json_object") ❺
     .build();        ❻

  ContactFinder finder =        ❼
   AiServices.create(ContactFinder.class, chatLanguageModel);

  String[] texts = {        ❽
     "John Doe, 123 Main St, Anytown, AZ 87654",
     "Mary Smith, 456 Cherry Lane, Someplace CA 90210",
     "John Henry Acme Widgets General Delivery Southampton MA",
  };

  for (String text : texts) {     ❾
   Contact ctc = finder.extractContactFrom(text);
   System.out.println(ctc);
  }
 }
}
```

❶ Nested Record or POJO class will hold the results of the conversion.

❷ `toString()` prints fields in JSON-like format.

❸ The interface to be implemented by the API, with `@UserMessage` telling the API in general what to look for, as if via a direct chat interface.

❹ The method we will call to parse the data into a `Contact` object.

❺ Builder pattern methods tell the API about the model we want to use, and that we want JSON format (which the LangChain4j documentation says works best).

❻ Builder `build()` method actually constructs the extractor object.

❼ Create the service implementation.

❽ Series of demo cases.

❾ For each demo, do the conversion and display the results.

When run, the preceding code prints the three contacts. The first two (not shown) are as expected; the third shows that the AI has done a decent job of separating the parts of the input despite the complete lack of commas. It won't always work, but it does in most common cases.

```
Contact {
      name    : John Henry
      company : Acme Widgets
      street  : General Delivery
      place   : Southampton, MA
}
```

And we are saved from the necessity of creating a JSON request and parsing a JSON object; the completed `Contact` object is returned from the invocation.

# 13.5 Conversing with Shadows

## Problem

Tired of one-way queries, you want to hold an ongoing conversation with an AI, similar to the web mode of ChatGPT.

## Solution

Use LangChain4j's `ChatMemory` to retain the context of a conversation.

## Discussion

The `MessageWindowChatMemory` is used to retain the context of a conversation. Used in conjunction with a service (as discussed in Recipe 13.4), it remembers the previous interactions with the AI, providing support for an ongoing conversation. In the following demo, we again ask for a list of the great literary themes, but then ask the AI to elaborate on two of them, not in numerical order, and see that the `ChatMemory` has given the AI context to expand on its previous answers. As usual, answers are ruthlessly edited to save paper.

```
>>> What are the great themes in literature?
Some of the great themes in literature include:

1. Love and relationships: This theme explores the complexities of human
relationships, including romantic love, family dynamics, and friendships.
2. Identity and self-discovery...
3. Power and corruption: Literature often explores the abuse of power and
the consequences of corruption in society.
4. War and conflict...
...
10. Freedom and liberation...

>>> Tell me more about the third item
Power and corruption is a recurring theme in literature that explores ...
Literature that explores power and corruption often features characters ...
By examining this theme, literature prompts readers to ...

>>> What more can you say about the first item
Love and relationships is a universal theme in literature that explores...
In literature, love and relationships are often used to explore themes of ...
Romantic love is a common focus in literature, with stories often centering...
Family relationships are also a rich source of material for exploring...
Friendships are an important aspect of the theme of love and relationships...
Overall, the theme of love and relationships in literature serves as...
```

The code that drove the preceding conversation is in Example 13-4. In real life, of course, the questions would be read interactively, either by a human typing or by another AI(!).

**Example 13-4. ai/src/main/java/langchain4j/Converse.java**

```
interface Chatterer {
  String speak(String response);
}

public class Converse {

  public static void main(String[] args) {

    String apiKey = Constants.getOpenAPIKey();
    ChatLanguageModel model = OpenAiChatModel.withApiKey(apiKey);

    Chatterer chatter = AiServices.builder(Chatterer.class)
      .chatLanguageModel(model)
      .chatMemory(MessageWindowChatMemory.withMaxMessages(10))
      .build();

    System.out.println(">>> " + Constants.TEXT_PROMPT);
    String list = chatter.speak(Constants.TEXT_PROMPT);
    System.out.println(list);
    System.out.println();

    final String moreOnThird = "Tell me more about the third item";
    System.out.println(">>> " + moreOnThird);
    String third = chatter.speak(moreOnThird);
    System.out.println(third);
    System.out.println();

    final String whosOnFirst = "What more can you say about the first item";
```

```
    System.out.println(">>> " + whosOnFirst);
    String first = chatter.speak(whosOnFirst);
    System.out.println(first);
  }
}
```

# 13.6 Generating Images with LangChain4j

## Problem

You want to generate images from prompts quickly and easily.

## Solution

Use LangChain4j's `ImageModel`.

## Discussion

The setup for generating images is quite simple with LangChain4j, as shown in Example 13-5. ChatGPT does not return images in the REST response, instead providing a URL to view the resulting image. The code extracts this URL and prints it. Then, if running in a desktop environment (e.g., not running in an application server), the program tries to display the resulting image in a browser. We could also read it using Java's `ImageIO`.

**Example 13-5. ai/src/main/java/langchain4j/LangChain4JImageMaker.java**

```
public static void main(String[] args) {

  String apiKey = Constants.getOpenAPIKey();
  ImageModel model = OpenAiImageModel.withApiKey(apiKey);
  Response<Image> response = model.generate(Constants.IMAGE_PROMPT);

  var respUrl = response.content().url();

  System.out.printf("Response URL is %s\n", respUrl);

  if (Desktop.isDesktopSupported()) {
    Desktop dtop = Desktop.getDesktop();
    if (dtop.isSupported(Desktop.Action.BROWSE)) {
          try {
             dtop.browse(URI.create(respUrl.toString()));
          } catch (IOException ex) {
       System.out.println("Unable to open, due to: " + ex);
          }
        } else {
      System.out.println("Desktop Open_URI Action not supported.");
    }
  } else {
    System.out.println("Desktop not supported");
  }
 }
}
```

As with most AI prompts, the results are nondeterministic or nonrepeatable. And sometimes contain hallucinations. Figure 13-3 shows some images that successive runs of this program generated, with the prompt "Charles Darwin in modern-day Toronto." The middle one bears little resemblance to the canonical images of my distant relative Sir Charles, but does feature some Victorian-era buildings alongside modernity. The last picture does look like the great naturalist, but if you blow up the picture, the child in a stroller with a tablet/phone is being pulled by a man with a detached left hand!



Figure 13-3. Generated images

# 13.7 Mixed Media Prompts: Inferences from Images with LangChain4j

## Problem

You want an AI to explain the contents of an image.

## Solution

Provide a LangChain4j `UserMessage` prompt containing both the image and a request for what needs explaining.

## Discussion

Some AI systems are capable of interpreting the contents of an image. For example, the image in Figure 13-4 might be explained as:

> *This image depicts a stylized, comic book-style illustration of a person standing in an urban environment. The person is wearing glasses and casual clothing, including a T-shirt and an open button-down shirt. They appear to be in a modern city setting, with tall skyscrapers and various buildings lining the street. The street is relatively empty, with a bus visible in the background. The sky is clear and blue, contributing to a bright and vibrant atmosphere. The overall aesthetic has a retro comic book feel.*



Figure 13-4. An image to explain

To get this description, the `UserMessage` prompt has to provide both the image data and a query such as "Tell me what you see in this image." LangChain4j provides its own `Image` class (not to be confused with the same-named class in java.awt). If the image resides on an external server, you can pass the image's URL. To provide the image directly, just read it into memory, base64 encode it, and pass that. This is illustrated in Example 13-6.

**Example 13-6. ai/src/main/java/langchain4j/LangChain4jImageInference.java**

```
/**
 * Simple demo of a chat-gpt text+media prompt and response.
 */
public class LangChain4jImageInference {

  public static void main(String[] args) {

    String apikey = Constants.getOpenAPIKey();

    ChatLanguageModel model = OpenAiChatModel.builder()
        .apiKey(apikey)
        .modelName(OpenAiChatModelName.GPT_4_O)
        .maxTokens(100)
```

```
        .build();

    var fileName = "mystery-image.png";
    byte[] imageData = null;
    try {
      imageData = Files.readAllBytes(Path.of(fileName));
    } catch (IOException e) {
      throw new RuntimeException(e);
    }
    Image image = Image.builder()
        .mimeType("image/png")
        .base64Data(Base64.getEncoder().encodeToString(imageData))
        .build();
    UserMessage userMessage = UserMessage.from(
        TextContent.from("Explain this image."),
        ImageContent.from(image)
    );

    Response<AiMessage> response = model.generate(userMessage);

    System.out.println(response.content().text());
  }
}
```

# 13.8 Running AI Locally with ollama

## Problem

You wish to run an LLM locally rather than in the cloud, perhaps for reasons of confidentiality.

## Solution

One solution is the ollama package, which can easily be installed for the three main operating systems (Linux, macOS, and Microsoft Windows).

## Discussion

To run ollama locally, you need to download the package and run it. Ollama will automatically use whatever GPU cores are on your machine; if no suitable GPUs are found, it will "fall back" to running (albeit slowly) on your CPU.

Start by fetching ollama from the project's download site. I installed it on macOS: just unzip the ZIP file into */Applications* or *~/Applications*. I also downloaded the `ollama` shell script to start the app running and interact with it. Then I typed `ollama run mistral`, which caused it to download the Mistral model (the black spot is the terminal-mode progress bar) and give me a prompt:

```
$ ollama run mistral
pulling manifest
pulling ff82381e2bea... 100% ▕▕▕▕▕▕▕▕▕▕▕▕▕▕▕▕ 4.1 GB
pulling (various other bits...)
verifying sha256 digest
writing manifest
removing any unused layers
success
>>> Send a message (/? for help)
```

```
>>> What are the great themes in literature?
There are countless themes that can be found in literature, but some of
the most common and universally relevant include:
1. Love and Relationships: ...
>>> /bye
$
```

There are many other LLM's available to be downloaded for use with ollama, listed at *https://ollama.com/library*.

Since my M2 Macintosh has adequate hardware, this started outputting almost immediately.

Of course we want to access this programmatically, so we can embed it into an application instead of running it interactively. LangChain4j has a solution! The toolkit will talk to ollama over a REST connection.

You need to add the langchain4j-ollama library to your *pom.xml* or *build.gradle*:

```
<dependency>
  <groupId>dev.langchain4j</groupId>
  <artifactId>langchain4j-ollama</artifactId>
  <version>${langchain4j-version}</version>
</dependency>
```

Example 13-7 shows accessing the ollama API and the Mistral model via the LangChain4j API.

**Example 13-7. ai/src/main/java/langchain4j/LangChain4JOllama.java**

```
import dev.langchain4j.model.chat.ChatLanguageModel;
import dev.langchain4j.model.ollama.OllamaChatModel;
import ai.Constants;

class LangChain4JOllama {

    public static void main(String[] args) {

        ChatLanguageModel model = OllamaChatModel.builder()
                .baseUrl("http://127.0.0.1:11434")
                .modelName("mistral")
                .build();

        String answer = model.generate(Constants.TEXT_PROMPT);

        System.out.println(answer);
    }
}
```

Since it's running on the same machine, "localhost" (IPv4 address 127.0.0.1) is the obvious URL hostname. And to be sure it's really running locally, I turned off both WiFi and wired network connections in `System Preferences`, and the program ran as before. The port number 11434 is commonly used by ollama, but if you're not sure, run the ollama program interactively with no arguments, and you'll see some debug chatter with a line similar to this:

```
time=2024-07-04T17:32:58.172-04:00 level=INFO source=routes.go:1111
  msg="Listening on 127.0.0.1:11434 (version 0.1.48)"
```

Running the access via REST appears to take longer, but that's only because the HTTP connection doesn't complete until the AI—which thinks it's talking to a person so it outputs at a human-readable speed—completes. If you want to watch it output, there is a "streaming mode" available in LangChain4j and most similar APIs.

There are many capabilities in the downloadable models; many of the features demonstrated earlier in the chapter (such as image inferencing) may not be available in all models. Either read the documentation for a given model, or just try it out and see if it works. Some models are general purpose; others are specialized, such as `codegemma` for software developers' coding tasks.

## See Also

Both LangChain4j and Spring AI are accompanied by a large set of example programs. The LangChain4j examples are at *https://github.com/langchain4j/langchain4j-examples/blob/main/open-ai-examples*. The official Spring AI examples are at *https://github.com/Azure-Samples/spring-ai-azure-workshop*, and a somewhat simplified version of the same material is at Ken Kousen's *https://github.com/kousen/springaiexamples*.

1 CUDA originally stood for Compute Unified Device Architecture.